

Week 5 Lab: Software Testing Fundamentals

From writing tests to designing effective testing

Student laboratory guide

Overview

In earlier ENSE707 labs you have already created MSTest projects, written unit/component tests, linked tests to requirements and acceptance criteria, and used test results as QA evidence. This lab deliberately moves beyond repeating those tasks. You will use the supplied Medical Appointment Booking System to decide what testing evidence is missing, derive tests systematically, inspect structural coverage, test at a larger system boundary, explore unexpected behaviour, and prioritise testing according to risk.

Key point: The central question is no longer only "Can I write a test?" It is "Which tests should we perform, at what level, using which technique, and why?"

Learning outcomes

- Classify existing testing evidence by test level, test type and test-design perspective.
- Apply equivalence partitioning and boundary-value analysis before writing test code.
- Use MSTest data-driven tests with DataRow to execute the same test logic with multiple inputs.
- Use code coverage as white-box evidence and explain why high coverage does not prove high software quality.
- Perform manual system tests through an external application interface rather than calling a service method directly.
- Conduct a structured exploratory testing session using a test charter and recorded observations.
- Prioritise further testing using product risk (likelihood and impact).
- Design and execute a small component integration test using real file persistence.
- Distinguish System Integration Testing (SIT), User Acceptance Testing (UAT) and Production Validation Testing (PVT).

Starter project and tools

Use the supplied Week 5 starter based on AppointmentBooking.zip. The baseline solution contains the AppointmentBooking class library and AppointmentBooking.Tests MSTest project. The supplied test suite contains 12 MSTest tests. The test assembly also enables method-level parallel execution.

- **Required tools:** Visual Studio, .NET, MSTest and Visual Studio Test Explorer (or run command line `dotnet test`).
- **Coverage tool:** Use Visual Studio code coverage if it is available in the lab installation. If it is not available, use the coverage output supplied by the lecturer rather than installing an unrelated third-party extension.
- **Source control:** Commit meaningful milestones to GitHub if your repository is being assessed for regular progress.

Required laboratory activities

Activity 1 - Establish the baseline and diagnose the existing suite

1. Open the supplied solution and build it without changing the production code.
2. Open Test Explorer and run all tests (or run command line `dotnet test`).
3. Record the number of tests discovered, passed and failed.
4. Skim the test names and identify the production classes they exercise.

5. Write at least four important behaviours or risks for which the current suite provides little or no evidence.

Discussion: A green test suite is evidence about the scenarios represented by the suite. It is not proof that the complete product is correct or ready for release.

Answer the following question in docs/Week5TestingMap.md: We have a passing automated test suite. What important kinds of confidence are still missing?

Activity 2 - Build a testing map of the existing project

Create docs/Week5TestingMap.md. Classify selected existing tests. You do not need to classify every test individually; choose at least six that represent different behaviours.

Existing test/evidence	Level	Type/focus	Technique/perspective	What it provides evidence for	Important gap
Doctor: negative slots	Your answer	Your answer	Your answer	Your answer	Your answer
Booking: no slots	Your answer	Your answer	Your answer	Your answer	Your answer
Patient: preferred display name	Your answer	Your answer	Your answer	Your answer	Your answer
Request: past appointment date	Your answer	Your answer	Your answer	Your answer	Your answer
Booking: helpful success message	Your answer	Your answer	Your answer	Your answer	Your answer
One other existing test of your choice	Your answer	Your answer	Your answer	Your answer	Your answer

Then add four rows labelled Integration, System, Acceptance and Non-functional evidence. State whether the current project already provides convincing evidence at each level/focus.

Important nuance: For some AppointmentBookingService tests, "unit" versus "small component test" can be arguable. Your classification is acceptable if you clearly state the test boundary and justify it.

Activity 3 - Design black-box tests before viewing the implementation

For this activity, begin from the behavioural rules below. Do not use the production source code to choose test data until you have completed the partitions and boundaries.

Rule set A - Doctor availability

- Available slots cannot be negative.
- A doctor with zero available slots cannot accept a booking.
- A doctor with one or more available slots can accept a booking.

Rule set B - Appointment date

- A requested date in the past is invalid.
- Today is valid in the current starter.
- A future date is valid.

Rule set C - Booking result

- A valid request with available capacity succeeds.
- A valid request with no available capacity fails.
- A successful booking consumes one available slot.
- Every booking attempt handled by AppointmentBookingService returns a clear BookingResult message.

In docs/BlackBoxTestDesign.md, create:

- [] Equivalence partitions for doctor slots, with at least one representative value from each partition.
- [] Boundary values around the change from invalid/unavailable/available slot counts.
- [] Equivalence partitions for appointment dates.
- [] Boundary values around the past/today transition.
- [] A small decision table for the BookAppointment operation.
- [] A list showing which designed cases already exist and which are missing from the baseline suite.

Use data-driven MSTest for selected boundaries

Add at least one data-driven MSTest using DataRow. The same test logic should run with several meaningful inputs derived from your test design, rather than from arbitrary values.

Example pattern - adapt it if your own design uses different cases

```
[TestMethod]
[DataRow(0, false)]
[DataRow(1, true)]
[DataRow(2, true)]
public void HasAvailableSlot_BoundaryCases(
    int availableSlots, bool expected)
{
    var doctor = new Doctor("D001", "Dr Mark", availableSlots);

    Assert.AreEqual(expected, doctor.HasAvailableSlot());
}
```

Add one additional data-driven test for appointment-date boundaries or invalid slot values. Run the full suite after adding your tests.

Activity 4 - White-box testing and structural coverage

Now inspect the production implementation. Look specifically for conditions, branches, exception paths and validation rules that are not represented by the existing tests.

1. Run code coverage for the test suite using the official Visual Studio coverage feature if available.
2. Identify at least two uncovered or weakly covered code paths.
3. Add targeted tests for those paths.
4. Run the full test suite again and rerun coverage.
5. Record what changed. Do not report a coverage percentage without explaining what new behaviour or structure the added test actually exercised.

Key point: Coverage tells you which code structures were executed. It does not tell you whether the requirements are correct, whether the assertions are strong, whether concurrency is safe, whether real integrations work, or whether users accept the product.

In docs/CoverageReview.md, answer:

- [] Which branch or path was initially missing?
- [] Which test did you add?
- [] What code became covered?
- [] What important quality risks are still not addressed even after coverage increases?
- [] Why would "100% line coverage" still be an insufficient release argument?

Activity 5 - Perform system testing through an external interface

The baseline automated tests call domain and service classes directly. For this activity, use a small console harness as the external interface of the complete prototype. The goal is not to learn console programming; it is to test from a larger system boundary.

Create AppointmentBooking.Console

1. Add a Console App project named AppointmentBooking.Console to the solution.
2. Add a project reference to AppointmentBooking.
3. Replace Program.cs with the starter code below.
4. Build and run the console application once before designing your system test records.

```
using ENSE707_AppointmentBooking;

Console.Write("Patient ID: ");
string patientId = Console.ReadLine() ?? "";

Console.Write("Legal name: ");
string legalName = Console.ReadLine() ?? "";

Console.Write("Preferred name (optional): ");
string preferredName = Console.ReadLine() ?? "";

Console.Write("Doctor name: ");
string doctorName = Console.ReadLine() ?? "";

Console.Write("Available slots: ");
int availableSlots = int.Parse(Console.ReadLine() ?? "0");

Console.Write("Days from today (-1, 0, 1, ...): ");
int daysFromToday = int.Parse(Console.ReadLine() ?? "0");

try
{
    var patient = new Patient(patientId, legalName, preferredName);
    var doctor = new Doctor("D001", doctorName, availableSlots);
    var request = new AppointmentRequest(
        patient, doctor, DateTime.Today.AddDays(daysFromToday));

    var service = new AppointmentBookingService();
    BookingResult result = service.BookAppointment(request);

    Console.WriteLine(result.Message);
    Console.WriteLine($"Remaining slots: {doctor.AvailableSlots}");
}
catch (Exception ex)
{
    Console.WriteLine($"Validation error: {ex.Message}");
}
```

Create manual system test records

Create docs/SystemTests.md. Execute at least the following four system scenarios through the console interface. Add one extra scenario of your own.

ID	Scenario	Key input	Expected result
SYS-01	Valid future booking with final available slot	Valid patient; slots = 1; days = 1	Success; preferred/legal display name is used; remaining slots = 0
SYS-02	No availability	Valid patient; slots = 0; days = 1	Failure message explains no available slots; remaining slots stays 0
SYS-03	Past appointment date	Valid patient; slots = 1; days = -1	Validation error; booking is not performed
SYS-04	Preferred name shown	Preferred name = Aroha; slots = 1; days = 1	Success message uses Aroha

For every executed system test, record Preconditions, Inputs, Steps, Expected Result, Actual Result, Pass/Fail and Evidence.

Reflection: Compare the system test with the existing MTest service tests. The difference is primarily the test boundary and entry point, not whether a human or a test framework presses the "run" button.

Activity 6 - Exploratory testing and risk-based prioritisation

Work from the following exploratory charter. Do not treat it as a fixed script.

Exploratory charter: Explore the appointment-booking workflow for ways that a booking could be duplicated, an available-slot count could become incorrect, or a receptionist could receive misleading feedback. Follow suspicious observations rather than only repeating the existing scripted cases.

Create docs/ExploratoryTesting.md and record your observations using a table similar to this:

Observation	Expected?	Evidence	Requirement clear?	Possible risk / next question
Example only: same request can be submitted twice	Your finding	Your evidence	Yes / No / Unclear	Your analysis
Your finding				
Your finding				

For at least three observations or plausible failure modes, assess product risk:

Risk	Likelihood	Impact	Priority	Testing response
Your risk	Low / Medium / High	Low / Medium / High	Your judgement	What evidence should be added?
Your risk				
Your risk				

Then answer:

- [] If you could add only one further test before release, which risk would you test first and why?
- [] Did exploratory testing reveal a confirmed defect, a requirement gap, a possible change request, or something that needs stakeholder clarification?
- [] Why is an unexpected behaviour not automatically a software defect?

Extension activities

Complete these activities after the required work if they are not finished in class. They extend the same Week 5 concepts to real integration boundaries and industry testing phases.

Activity 7 - Component integration testing with real persistence

Unit tests in the baseline project keep all state in memory. This extension introduces a real file-system dependency so that you must manage test data, environment state and cleanup.

Create AppointmentBooking.Persistence

1. Add a Class Library project named AppointmentBooking.Persistence.
2. Add a project reference to AppointmentBooking.
3. Create BookingRecord.cs, JsonBookingRepository.cs and PersistentAppointmentBookingService.cs using the supplied starter code.
4. Add a new MSTest project named AppointmentBooking.IntegrationTests with references to AppointmentBooking and AppointmentBooking.Persistence.

BookingRecord.cs

```
namespace AppointmentBooking.Persistence;

public record BookingRecord(
    string PatientId,
    string DoctorId,
    DateTime RequestedDate,
    string Message);
```

JsonBookingRepository.cs

```
using System.Text.Json;

namespace AppointmentBooking.Persistence;

public class JsonBookingRepository
{
    private readonly string _filePath;

    public JsonBookingRepository(string filePath)
    {
        _filePath = filePath;
    }

    public void Add(BookingRecord record)
    {
        var records = GetAll().ToList();
        records.Add(record);
        File.WriteAllText(
            _filePath,
            JsonSerializer.Serialize(records));
    }

    public IReadOnlyList<BookingRecord> GetAll()
    {
        if (!File.Exists(_filePath))
            return Array.Empty<BookingRecord>();

        string json = File.ReadAllText(_filePath);
        return JsonSerializer.Deserialize<List<BookingRecord>>(json)
            ?? new List<BookingRecord>();
    }
}
```

PersistentAppointmentBookingService.cs

```

using ENSE707_AppointmentBooking;

namespace AppointmentBooking.Persistence;

public class PersistentAppointmentBookingService
{
    private readonly AppointmentBookingService _bookingService;
    private readonly JsonBookingRepository _repository;

    public PersistentAppointmentBookingService(
        AppointmentBookingService bookingService,
        JsonBookingRepository repository)
    {
        _bookingService = bookingService;
        _repository = repository;
    }

    public BookingResult BookAppointment(AppointmentRequest request)
    {
        BookingResult result = _bookingService.BookAppointment(request);

        if (result.Success)
        {
            _repository.Add(new BookingRecord(
                request.Patient.Id,
                request.Doctor.Id,
                request.RequestedDate,
                result.Message));
        }

        return result;
    }
}

```

Write at least two integration tests:

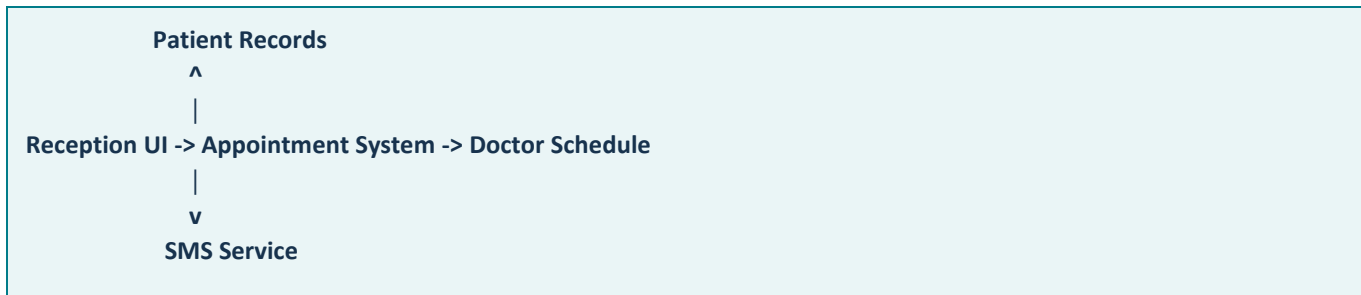
- [] A successful booking is persisted and can be read by a fresh repository instance.
- [] A failed booking is not persisted.

Test isolation: The supplied starter has method-level parallel test execution enabled. Do not make all integration tests share the same test-data filename. Use a unique temporary file for each test and clean it up after the test.

In docs/IntegrationTesting.md, explain why these tests are not equivalent to the existing in-memory service tests. Discuss real I/O, environment state, cleanup, isolation, speed and possible failure causes.

Activity 8 - Design SIT, UAT and PVT scenarios

The Week 2 problem statement says that the clinic may later integrate the booking system with patient records, doctor schedules, SMS reminders and online booking. Use the future architecture below as the basis for this activity.



Create docs/IndustryTestingPhases.md. Propose at least two scenarios for each phase:

Phase	Purpose in this future system	Your scenarios should focus on
SIT	Check interaction between complete systems/services	API/data mapping, authentication, timeouts, retries, external failures, end-to-end data flow
UAT	Determine whether real business users can accept the workflow	Reception tasks, clear decisions/messages, business rules, operational usefulness
PVT	Validate the deployed production configuration safely	Production endpoints/configuration, controlled accounts/data, connectivity, logs/monitoring, critical smoke checks

Also answer: Why can the existing 12 MSTest tests not simply be renamed "UAT" or "PVT"?

Optional challenge - Concurrency risk

Inspect `Doctor.HasAvailableSlot()`, `Doctor.ReserveSlot()` and `AppointmentBookingService.BookAppointment()`. Assume one slot remains and two booking requests execute concurrently.

- [] Draw one possible interleaving in which both requests observe an available slot before the decrement is safely resolved.
- [] State possible outcomes, including an incorrect final slot count or an exception from one request.
- [] Explain why repeatedly running a naïve Task-based test is not strong proof that the code is thread-safe.
- [] Propose a production-level mitigation and a stronger type of evidence (for example, an atomic database transaction/constraint plus concurrency-focused integration testing).

Lab deliverables

- [] Updated AppointmentBooking solution with the new Week 5 tests.
- [] docs/Week5TestingMap.md
- [] docs/BlackBoxTestDesign.md
- [] docs/CoverageReview.md
- [] docs/SystemTests.md
- [] docs/ExploratoryTesting.md
- [] Extension: integration-test project and docs/IntegrationTesting.md
- [] Extension: docs/IndustryTestingPhases.md
- [] Meaningful Git commits if required by your course/project process.

Official Microsoft testing references

MSTest overview: <https://learn.microsoft.com/en-us/dotnet/core/testing/unit-testing-mstest-intro>

Data-driven testing in MSTest: <https://learn.microsoft.com/en-us/dotnet/core/testing/unit-testing-mstest-writing-tests-data-driven>

Run tests with Visual Studio Test Explorer: <https://learn.microsoft.com/en-us/visualstudio/test/run-unit-tests-with-test-explorer>

Visual Studio code coverage: <https://learn.microsoft.com/en-us/visualstudio/test/using-code-coverage-to-determine-how-much-code-is-being-tested>